

Project: Containerized CLI Login System with Optional 2FA

Objective

Build a secure **command-line login system** that supports user registration, authentication, optional **TOTP-based 2FA**, and session management. The system must run in Docker containers and use a database for persistence.

This assignment will test your ability to:

- Work with Go
- Implement authentication and security best practices.
- Use Docker for containerization.
- Design clean, maintainable code.
- Build an interactive CLI.

Requirements

1. Authentication System

- User **registration** with username and password.
- **Login** with username and password.
- Optional **TOTP-based 2FA** (Google Authenticator compatible).
- **Secure password storage** (hashing with bcrypt or similar).
- **Account lockout** after multiple failed attempts.
- **Session management** with configurable timeout.

2. Database Integration Choose **one** :

- SQLite (recommended for simplicity)
- MySQL
- PostgreSQL

The database must:

- Run in a container.
- Persist data across container restarts.

3. Command-Line Interface

- Interactive prompt with **history** and **tab-completion**.
- Clear **error messages** and **success feedback**.

- help command to list available commands.

4. Commands **Before Login:**

- register → create a new user
- login → login with username/password (+ TOTP if enabled)
- help → show available commands
- exit → quit program

After Login:

- whoami → show current user details
- enable-2fa → enable TOTP-based MFA
- disable-2fa → disable MFA
- logout → end session
- help → show available commands

5. User Details (Auto-display After Login)

- Username
- Registration date
- MFA status (enabled/disabled)
- Session expiration time
- Last login time (if available)



Deliverables

- **Source Code** (well-structured, with comments).
- **Dockerfile** + **docker-compose.yml** to run the project.
- **README.md** with setup instructions and usage guide.
- **Database schema/migrations** included.
- (Optional but nice) Unit tests for core functionality.



Evaluation Criteria

- Correctness (does it meet all requirements?)
- Code quality (readability, structure, error handling).
- Security (password hashing, session handling, lockouts).

- Use of Docker (clean setup, persistence).
- Usability (clear CLI, helpful feedback).
- Documentation (README completeness).



Submission Instructions - Email at hr@osto.one

- Push your code to a **GitHub/GitLab repository** (public or private with access).
- Include a README with:
- Share the repository link when complete.